



ECOLE MAROCAINE DES
SCIENCES DE L'INGENIEUR

Membre de 
HONORIS UNITED UNIVERSITIES

Sécurité des applications

2025/2026

Plan du module:

Chapitre 1 : Bases de la sécurité

Chapitre 2 : Sécurité du client

Chapitre 3 : Sécurité du réseau

Chapitre 4 : Architecture des serveurs Web

Liens Utiles

<https://owasp.org/www-project-application-security-verification-standard/>

https://wiki.mozilla.org/Security/Server_Side_TLS

<https://www.cloudflare.com/th-th/learning/ssl/what-is-https/>

<https://www.cloudflare.com/fr-fr/learning/ssl/what-happens-in-a-tls-handshake/>

<https://aaronparecki.com/oauth-2-simplified/>

<https://docs.faveohelpdesk.com/docs/helpers/server-hardening/nginx-hardening>

https://cheatsheetseries.owasp.org/cheatsheets/Logging_Cheat_Sheet.html

Plateforme des TP:

Logiciels à installer:

- ✓ Wireshark.
- ✓ Nmap.
- ✓ OWASP ZAP
- ✓ Docker

Machines virtuelles

- ✓ Ubuntu Server / Ubuntu Desktop.
- ✓ Kali Linux.
- ✓ Metasploitable 2.
- ✓ DVWA
- ✓ Optionnel : serveur Apache avec ModSecurity (WAF).

Les outils et Environnement des Travaux pratiques

	Outil	Rôle	Installation
01	Docker Desktop	Conteneurisation — lancer Juice Shop	Windows/Mac/Linux
02	OWASP Juice Shop	Application web vulnérable cible	Via Docker
03	Kali Linux (VM)	OS de securite — tous outils preinstallés	VirtualBox + ISO
04	Burp Suite Community	Proxy HTTP, Repeater, Intruder	Windows/Kali
05	Firefox + DevTools	Navigateur + inspection HTTP/DOM/cookies	Inclus dans Kali
06	OWASP ZAP	Scanner automatique de vulnérabilités	Windows/Kali
07	Metasploitable 2	Tester le reseau et services(FTP,SSH,SMB,MYSQL)	Windows/Kali
08	WebGoat	Application web Java vulnérable guidée	Docker
09	Nmap	Scan de ports et services réseau	Kali (preinstalle)
10	Wireshark	Analyse de trafic réseau — captures TLS/HTTP	Kali (preinstalle)
11	SQLMap	Automatisation des injections SQL	Kali (preinstalle)
12	Python 3 + bcrypt	Scripts de hachage et cryptographie	Kali (preinstalle)
	Outils en ligne	CrackStation, jwt.io, webhook.site	Aucune — navigateur

CHAPITRE 1

BASES DE LA SÉCURITÉ

Architecture, Same-Origin Policy et attaques côté client

Objectifs:

- Définir et distinguer les concepts fondamentaux de la sécurité informatique
- Appliquer la triade CIA a des situations réelles d'ingenierie logicielle
- Identifier et classifier les menaces, vulnérabilités et risques informatiques
- Connaitre et expliquer le Top 10 OWASP 2021
- Appliquer les principes de conception sécurisée dans le développement
- Utiliser les outils de base de l'audit de sécurité (Burp Suite, Juice Shop)

Pourquoi la sécurité est-elle critique en 2025 ?

Indicateur	Chiffre	Source / Impact
Cout moyen d'une violation de données	4,88 millions USD	IBM Cost of a Data Breach Report 2024
Temps moyen pour détecter une intrusion	194 jours	IBM Security 2024
Augmentation des cyberattaques	+300% depuis 2020	ANSSI / Interpol
Part des attaques visant les applications web	43%	Verizon DBIR 2024
Entreprises touchées par un ransomware	66% en 2023	Sophos State of Ransomware 2024
Proportion de failles dues au code applicatif	91%	OWASP / NIST Statistics

La surface d'attaque d'une application web

Composant	Points d'attaque potentiels
Couche Présentation	Formulaires HTML, champs de saisie, URLs, parametres GET/POST, fichiers uploads, cookies, headers HTTP
Couche Logique Métier	APIs REST/SOAP, logique d'autorisation, flux de travail, gestion des sessions, règles métier
Couche Donnees	Requêtes SQL/NoSQL, procédures stockées, ORM, fichiers de configuration, logs
Infrastructure	Serveur web, serveur d'application, base de données, système de fichiers, services tiers, dépendances
Authentification	Mécanisme de login, réinitialisation de mot de passe, 2FA, gestion des tokens, single sign-on

Les niveaux de sécurité

Sécurité réseau : dans ce cas, on cherche à instaurer la sécurité au niveau d'un réseau (ou sous-réseau) entier de telle sorte à empêcher toute atteinte à n'importe quel élément faisant partie de ce réseau là. Les firewalls et les NIPS (Network-based Intrusion Prevention System) sont des solutions souvent envisagées dans cette situation.

Les niveaux de sécurité (suite)

Sécurité système :

dans ce cas de figure, le système et les services tournés sur un ordinateur (serveur, station de travail, terminal...) est ciblé par la sécurité.

Les antivirus constituent une solution très populaire, mais aussi les HIPS (Host-based Intrusion Prevention System) ou HIDS (Host-based Intrusion Detection System).

Les niveaux de sécurité (suite)

Sécurité applicative : Ici ce n'est ni le réseau ni le système entier d'un ordinateur qui nous intéresse, mais un logiciel ou une application en particulier (site, application web...). Dans ce cas de figure on peut utiliser des correctifs, des filtres, des IPS applicatifs, des WAF (Web Application Firewall)...

La sécurité informatique

- Sécurité réseau
- Sécurité système
- Sécurité applicative
- Sécurité system embarque (IoT, ffd)
- Sécurité BIGDATA
- Sécurité IA Machine learning deep learning

Sécurité réseau

- Protège **l'ensemble d'un réseau ou sous-réseau**
- Objectif : empêcher les attaques externes ou internes
- Agit au niveau du **trafic réseau**
- Contrôle des flux entrants et sortants

Solutions principales :

- **Firewall** (pare-feu)
- **NIPS** (Network Intrusion Prevention System)
- **IDS** Intrusion Detection System **réseau VPN sécurisés**

Sécurité système

- Protège **un ordinateur ou serveur spécifique**
- Sécurise le **système d'exploitation et les services**
- Vise à empêcher les attaques locales ou distantes
- Contrôle des processus et fichiers

Solutions principales :

- Antivirus / Antimalware
- **HIPS** (Host-based Intrusion Prevention System)
- **HIDS** (Host-based Intrusion Detection System)
- **Gestion des mises à jour et correctifs**

Sécurité applicative

- Protège **une application précise (site web, logiciel, API...)**
- Cible les vulnérabilités logicielles
- Vise à prévenir les attaques comme **SQL injection, XSS, etc.**

Solutions principales :

- **Correctifs** (patch management)
- **WAF** (Web Application Firewall)
- **IPS** (Intrusion Prevention System) applicatifs
- **Filtrage** et **validation** des entrées

La Triade CIA (*Confidentialité, Intégrité, Availability*)

- La triade **CIA** :
(Confidentiality, Integrity, Availability) est le modèle fondateur de la sécurité de l'information.
- Formule dans les années 1970, ce modèle reste aujourd'hui la référence universelle pour évaluer et concevoir des systèmes sécurisés.



La Triade CIA (*Confidentialité, Intégrité, Availability*)

C: Confidentialité	I :Intégrité	A :Disponibilité
Définition : Garantir que l'information n'est accessible qu'aux personnes, processus ou systèmes autorisés. Menaces : Ecoute réseau, vol de credentials, accès non autorise, exposition de données en clair. Contre-mesures : Chiffrement (AES, TLS), authentification forte, contrôle d'accès (RBAC), gestion des privilèges. Exemple concret : Un dossier médical ne doit être lisible que par le médecin traitant, pas par un employé administratif.	Définition : Garantir que les données sont exactes, complètes et n'ont pas été modifiées de manière non autorisée. Menaces : Falsification de données, injection SQL, man-in-the-middle, corruption de fichiers. Contre-mesures : Signatures numériques, hachage (SHA-256), journaux d'audit, validation des entrées. Exemple concret : Un virement bancaire de 1000 DZD ne doit pas pouvoir être modifié en transit pour devenir 100 000 DZD.	Définition : Garantir que les systèmes, services et données sont accessibles et utilisables quand les utilisateurs en ont besoin. Menaces : Attaques DDoS, pannes matérielles, ransomware, erreurs de configuration. Contre-mesures : Redondance, sauvegardes, plans de reprise d'activité (PRA/PCA), load balancing. Exemple concret : Un hôpital doit accéder aux dossiers patients 24h/24. Une panne de système peut coûter des vies.

Extension : le modèle AAA

La triade CIA est souvent complétée par le modèle AAA (Authentication, Authorization, Accounting) pour couvrir la gestion des accès. Ces deux modèles sont complémentaires et forment ensemble un cadre complet de sécurité.

Propriété AAA	Description détaillée
Authentication (Authentification)	Processus de vérification de l'identité d'un utilisateur ou d'un système. Répond à la question : 'Qui êtes-vous ?' Méthodes : mot de passe, certificat, biométrie, token OTP, 2FA/MFA. La faiblesse de l'authentification est la cause n°1 des violations de données.
Authorization (Autorisation)	Une fois authentifié, détermination de ce que l'utilisateur a le droit de faire. Répond à la question : 'Que pouvez-vous faire ?' Mécanismes : RBAC (Role-Based Access Control), ABAC (Attribute-Based), ACL (Access Control List). Principe fondamental : moindre privilège.
Accounting (Imputabilité)	Enregistrement et surveillance des actions des utilisateurs pour audit et traçabilité. Répond à la question : 'Qu'avez-vous fait ?' Outils : journaux système (logs), SIEM, audit trails. Essentiel pour la détection d'intrusions et la réponse aux incidents.

Autres propriétés de sécurité complémentaires

Au-delà de [CIA](#) et [AAA](#), les systèmes modernes doivent également garantir des propriétés additionnelles reconnues par [les normes ISO 27001](#) et [NIST](#) :

Propriété	Définition et importance
Non-répudiation	Garantir qu'une personne ne peut pas nier avoir réalisé une action. Essentiel dans les transactions électroniques, contrats numériques et preuves judiciaires. Repose sur les signatures numériques.
Authenticité	Garantir que les données proviennent bien de la source prétendue et n'ont pas été falsifiées en transit. Complémentaire à l'intégrité.
Fiabilité (Reliability)	Le système se comporte de manière cohérente et prévisible, même face à des entrées inattendues ou malveillantes.
Résilience	Capacité du système à continuer à fonctionner (peut-être de manière dégradée) même après une attaque ou une panne partielle.
Vie privée (Privacy)	Protection des données personnelles et respect du droit à la vie privée. Dimension juridique (RGPD, loi 18-07) en plus de la dimension technique.

Menaces, Vulnérabilités et Risques

Ces quatre concepts sont au cœur de toute démarche de sécurité. Ils sont souvent confondus, mais leurs distinctions sont essentielles pour communiquer efficacement et prendre les bonnes décisions.

Concept	Définition précise
Menace (Threat)	Evènement ou acteur susceptible de causer un dommage au système. Une menace est potentielle : elle peut se matérialiser ou non. Exemples : un hacker, un employé malveillant, une catastrophe naturelle, une coupure de courant.
Vulnérabilité (Vulnerability)	Faiblesse dans un système, une application ou un processus qui peut être exploitée par une menace. Une vulnérabilité est une caractéristique interne du système. Exemples : un mot de passe faible, un code non valide, une version de logiciel obsolète.
Attaque (Attack)	Action concrète menée par une menace pour exploiter une vulnérabilité. L'attaque est la matérialisation de la menace sur la vulnérabilité. Exemples : injection SQL exécutée, phishing envoyé, force brute lancée.
Risque (Risk)	Estimation de la probabilité qu'une menace exploite une vulnérabilité et de l'impact résultant. C'est la notion qui guide les décisions de sécurité. Formule : $\text{Risque} = \text{Probabilité} \times \text{Impact}$.
Contre-mesure (Countermeasure)	Action ou mécanisme mis en place pour réduire un risque. Peut agir sur la probabilité (prévention) ou l'impact (mitigation). Exemples : patch de sécurité, pare-feu, formation des employés.
Actif (Asset)	Tout ce qui a de la valeur pour l'organisation et qui nécessite une protection. Inclut les données, logiciels, matériel, personnel, réputation, propriété intellectuelle.

La formule du risque et la matrice

La gestion des risques repose sur une évaluation quantitative ou qualitative combinant deux dimensions :

$$\text{RISQUE} = \text{Probabilité} \times \text{Impact}$$

Taxonomie des attaques

Les attaques informatiques se classifient selon plusieurs dimensions complémentaires.
Cette classification aide a choisir les bonnes contre-mesures

Classification par nature

Type	Description et exemples
Attaques passives	L'attaquant observe et collecte des informations sans modifier les systèmes ou les données. Difficiles a détecter car elles ne laissent pas de traces évidentes. Exemples : écoute réseau (sniffing), analyse de trafic, reconnaissance OSINT (Open Source Intelligence).
Attaques actives	L'attaquant modifie, perturbe ou détruit des systèmes et données. Plus facilement détectables mais généralement plus destructrices. Exemples : injection SQL, déni de service, modification de données, usurpation d'identité.

Classification par origine

Origine	Description
Attaques externes	Menaces provenant de l'extérieur du réseau ou de l'organisation. L'attaquant doit d'abord pénétrer le périmètre de sécurité. Exemples : hacker externe, nation-state actor, cybercriminel.
Attaques internes	Menaces émanant de l'intérieur : employé malveillant, sous-traitant, stagiaire ayant accès au système. Particulièrement dangereuses car l'attaquant contourne les défenses de périmètre.
Attaques par rebond	L'attaquant utilise un système intermédiaire compromise (pivot) pour attaquer une cible finale. Complexifie l'attribution et contourne les restrictions réseau.

Les vecteurs d'attaque les plus courants

Un vecteur d'attaque est le chemin ou le moyen utilise par un attaquant pour accéder a un système cible. Voici les vecteurs les plus fréquemment exploites dans les applications web :

Vecteur d'attaque	Description détaillée et mécanisme
Injection (SQL, NoSQL, OS, LDAP)	Insertion de code malveillant dans une entrée de l'application non validée. Le code est alors interprète et exécute par le système cible. Cause n 3 de l'OWASP Top 10. Exemple : ' OR '1'='1 dans un formulaire de login.
Cross-Site Scripting (XSS)	Injection de scripts JavaScript malveillants dans des pages web vues par d'autres utilisateurs. Permet de voler des cookies de session, rediriger vers des sites malveillants, ou réaliser des actions a la place de la victime.
Authentification compromise	Exploitation des mécanismes d'authentification faibles : mots de passe par défaut, sessions non expirées, tokens prédictibles, absence de 2FA. Donne directement accès aux comptes utilisateurs.
Phishing et ingenierie sociale	Manipulation psychologique pour obtenir des credentials ou installer des malwares. Reste le vecteur d'entrée initial le plus efficace malgré les défenses techniques.
Man-in-the-Middle (MitM)	Interception des communications entre deux parties. L'attaquant peut lire et/ou modifier les données en transit. Facilite sur les reseaux WiFi publics non chiffres.
Supply Chain Attack	Compromission d'une dépendance tierce (librairie, outil de build, fournisseur) pour introduire du code malveillant dans de nombreuses applications. Exemple : incident SolarWinds 2020.
Deni de Service (DoS/DDoS)	Saturation d'un service pour le rendre indisponible. Les attaques DDoS impliquent des centaines de milliers de machines (botnet). Cible principalement la propriété Disponibilite de la triade CIA.
Exploitation de vulnérabilités (CVE)	Utilisation de failles connues et répertoriées dans des logiciels, frameworks ou composants. Efficace contre les systèmes non mis a jour. Source : https://cve.mitre.org/

Cycle de vie d'une attaque : la Kill Chain

Le concept de Cyber Kill Chain, développé par Lockheed Martin, décrit les étapes successives d'une attaque avancée. Comprendre ce modèle permet d'anticiper et de bloquer les attaques a différents stades :

	Phase	Description	Detection / Blocage
1	Reconnaissance	Collecte d'informations sur la cible (OSINT, scan de ports, analyse DNS).	Reduire l'exposition publique, surveiller les scans entrants
2	Armement	Creation ou adaptation d'un exploit adapte aux vulnerabilites decouvertes.	Patch management, veille CVE
3	Livraison	Envoi du payload vers la cible (email phishing, USB, exploit web).	Filtrage email, formation utilisateurs, WAF
4	Exploitation	Execution du code malveillant sur le systeme cible.	SAST/DAST, sandboxing, EDR
5	Installation	Installation d'une backdoor pour maintenir l'acces persistant.	Monitoring des processus, integrité systeme
6	Command & Control	Etablissement d'un canal de communication avec l'attaquant.	Filtrage DNS, detection anomalies reseau
7	Actions sur objectifs	Exfiltration, chiffrement, destruction selon les objectifs.	DLP, alertes SIEM, sauvegardes offline

OWASP Top 10 : 2021 (Référence mondiale)

OWASP (Open Web Application Security Project) est une fondation internationale a but non lucratif fondée en 2001. Sa mission est d'améliorer la sécurité des logiciels.

Le Top 10 OWASP est son document de référence le plus connu : il classe les 10 catégories de vulnérabilités web les plus critiques, mis a jour tous les 3 a 4 ans sur la base de données réelles collectées auprès de milliers d'organisations.

La version 2021 est la référence actuelle. Elle a introduit des changements importants par rapport a la version 2017, notamment l'élévation des problèmes de conception (A04) et des attaques sur la chaine d'approvisionnement (A08).

OWASP Top 10 :2021 Tableau de synthèse

	Catégorie	Type de faille	Protection principale
A01	Contrôle d'accès défaillant	IDOR, Privilège Escalation	Vérifier autorisations cote serveur
A02	Défaillances cryptographiques	Données exposées, hashage faible	HTTPS, bcrypt, AES-256
A03	Injection	SQL, NoSQL, OS, LDAP	Requetes preparees, validation
A04	Conception non sécurisée	Logique métier faillible	Threat Modeling, SDL
A05	Mauvaise configuration	Services inutiles, defaults	Hardening, suppression defaults

OWASP Top 10 :2021 Tableau de synthèse (Suite)

	Catégorie	Type de faille	Protection principale
A06	Composants vulnérables	CVE, bibliothèques obsolètes	SBOM, scan automatique
A07	Authentification défectueuse	Brute force, sessions faibles	MFA, bcrypt, politique MP
A08	Intégrité logiciels/données	Supply chain, désérialisation	Signatures, CI/CD sécurisé
A09	Journalisation insuffisante	Intrusions non détectées	SIEM, alertes, logs complets
A10	SSRF	Accès ressources internes	Whitelist URLs, firewall

Principes de Conception Sécurisée

Les bonnes pratiques du développement

La sécurité des applications ne se résume pas à corriger des bugs après coup. Elle doit être une composante fondamentale de la démarche de développement. Les principes présentes dans cette section sont les fondements sur lesquels repose toute architecture sécurisée moderne.

Les 8 principes de Salter et Schroeder (1975)

P1	Moindre privilège (Least Privilege) Chaque composant du système (utilisateur, processus, programme) ne doit disposer que des droits strictement nécessaires à l'accomplissement de sa fonction, et rien de plus. Ce principe limite les dommages en cas de compromission. Exemple : Un serveur web doit pouvoir lire les fichiers statiques mais jamais modifier la configuration du système. Un utilisateur standard ne doit pas avoir accès aux données des autres utilisateurs.
P2	Défense en profondeur (Defense in Depth) Mettre en place plusieurs couches de sécurité indépendantes. Si une couche est contournée, les autres continuent de protéger le système. Ne jamais reposer sur un seul mécanisme de sécurité. Exemple : Un attaquant qui contourne le WAF doit encore faire face à la validation des entrées, aux requêtes préparées, au contrôle d'accès applicatif et aux restrictions de la base de données.

Principes de Conception Sécurisée (Suite)

Les bonnes pratiques du développement

P3	Echec sécurise (Fail Secure / Fail Safe) Quand un système échoue ou rencontre une erreur, il doit se retrouver dans un état sécurisé par défaut. L'accès doit être refusé en cas de doute, pas accordé. Exemple : Si le système d'autorisation est indisponible, l'accès doit être refusé (fail secure), non accordé (fail open). Un portillon d'accès défaillant doit se verrouiller, pas s'ouvrir.
P4	Economie des mécanismes (Economy of Mechanism) Privilégier la simplicité dans la conception des mécanismes de sécurité. Les systèmes simples sont plus faciles à analyser, tester et maintenir. La complexité est l'ennemie de la sécurité. Exemple : Un système d'authentification simple avec 2FA bien implémenté est plus sûr qu'un système complexe avec 10 facteurs d'authentification mal implémentés et difficiles à auditer.
P5	Médiation complète (Complete Mediation) Chaque accès à chaque ressource doit être valide à chaque tentative. Ne jamais mettre en cache les décisions d'autorisation de manière non sécurisée, ni présumer que les droits n'ont pas changé depuis la dernière vérification. Exemple : Vérifier les permissions à chaque requête API, même si l'utilisateur vient de les vérifier 30 secondes avant. Un utilisateur peut être révoqué entre deux requêtes.

Principes de Conception Sécurisée (Suite)

Les bonnes pratiques du développement

P6	Conception ouverte (Open Design) La sécurité ne doit pas reposer sur le secret de l'implémentation (security through obscurity). L'algorithme doit être public, seules les clés doivent rester secrètes (principe de Kerckhoffs). Exemple : AES est un algorithme de chiffrement public, largement analyse et éprouve. Sa sécurité repose sur la clé, pas sur le secret de l'algorithme. Contrairement a un algorithme maison non analyse.
P7	Separation des privileges (Separation of Privilege) Une operation critique doit necessiter l'accord de plusieurs conditions ou entites independantes pour etre autorisee. Reduit le risque qu'une seule compromission suffise. Exemple : Pour effectuer un virement bancaire important, requierir a la fois l'authentification par mot de passe ET la validation par un code SMS envoye sur le telephone enregistre.
P8	Mecanismes peu intrusifs (Least Common Mechanism) Minimiser le nombre de mecanismes partages entre plusieurs utilisateurs. Le partage cree des canaux potentiels de fuite d'information ou d'attaque entre utilisateurs. Exemple : Eviter que plusieurs clients partagent le meme processus serveur sans isolation. Utiliser des conteneurs ou des espaces memoire separees pour isoler les clients les uns des autres.

Le Secure Development Lifecycle (SDL)

SDL (Secure Development Lifecycle) est une approche de développement logiciel qui intègre la sécurité a chaque phase du cycle de vie. Originellement développe par Microsoft en 2004, il est aujourd'hui adopté par la majorité des grandes organisations.

Phase	Activites SDL	Outils / Methodes	Livrables
Formation	Formation sécurité des équipes de dev	OWASP Top 10, Secure Coding Guidelines	Equipe formee et certifiee
Exigences	Définition des exigences de sécurité et de conformité	Threat Modeling (STRIDE), Privacy Impact Assessment	Security Requirements Document
Conception	Revue d'architecture orientée sécurité, modélisation des menaces	Diagrammes de flux de donnees, revues de paires	Architecture securisee validee
Implementation	Codage sécurise, revues de code, outils d'analyse statique	SAST (SonarQube, Checkmarx), Secure Coding Standards	Code revue et conforme
Verification	Tests de securite : pentest, fuzzing, revue de configuration	DAST (OWASP ZAP, Burp Suite), tests d'intrusion	Rapport de vulnérabilités
Deploiement	Configuration securisee, hardening, gestion des secrets	Chef, Ansible, Vault (HashiCorp), IaC	Environnement durci
Maintenance	Surveillance, patch management, reponse aux incidents	SIEM, CVE monitoring, Dependabot	Système a jour et surveille

DevSecOps :La sécurité dans les pipelines modernes

DevSecOps est l'évolution naturelle du **DevOps** qui intègre la sécurité ('Sec') directement dans les pipelines d'intégration et de déploiement continus (**CI/CD**). L'objectif est de détecter et corriger les vulnérabilités le plus tôt possible dans le cycle de développement

Pratique DevSecOps	Description et mise en œuvre
SAST (Static Application Security Testing)	Analyse du code source sans l'exécuter pour détecter les vulnérabilités. Intègre dans l'IDE et/ou le pipeline CI. Outils : SonarQube, Checkmarx, Semgrep, Bandit (Python). Détecte : injections, secrets en dur dans le code, mauvaises pratiques cryptographiques.
DAST (Dynamic Application Security Testing)	Test de l'application en cours d'exécution pour détecter les vulnérabilités comportementales. Simule des attaques réelles. Outils : OWASP ZAP, Burp Suite Enterprise. Détecte : XSS, injections, problèmes d'authentification.
SCA (Software Composition Analysis)	Analyse des dépendances tierces pour identifier les composants vulnérables. Outils : OWASP Dependency-Check, Snyk, Dependabot. Détecte : CVE dans les bibliothèques, licences incompatibles.
Secrets Detection	Détection des secrets (clés API, mots de passe, tokens) accidentellement commises dans le code source. Outils : GitLeaks, TruffleHog, Git-secrets. Particulièrement important avant les pushes sur les dépôts publics.
Infrastructure as Code Security	Analyse des fichiers de configuration d'infrastructure (Terraform, CloudFormation, Kubernetes YAML) pour détecter les mauvaises configurations. Outils : Checkov, tfsec, kube-bench.
Container Security	Analyse des images Docker pour détecter les vulnérabilités dans les couches de base et les paquets installés. Outils : Trivy, Clair, Docker Scout. Essentiel dans les architectures microservices.

Bonnes pratiques de codage sécurisé

Au niveau du code, voici les règles essentielles que tout développeur doit appliquer systématiquement :

Catégorie	Bonnes pratiques a appliquer
Validation des entrées	Ne jamais faire confiance aux données utilisateur. Valider cote serveur systematiquement (pas seulement cote client). Utiliser des listes blanches (ce qui est autorise) plutot que des listes noires (ce qui est interdit). Valider le type, la longueur, le format et la plage de chaque entrée.
Gestion des secrets	Ne JAMAIS hardcoder des mots de passe, cles API ou tokens dans le code source. Utiliser des variables d'environnement ou des gestionnaires de secrets (HashiCorp Vault, AWS Secrets Manager). Vérifier le .gitignore avant chaque commit.
Gestion des erreurs	Ne pas exposer les détails techniques des erreurs aux utilisateurs finaux (stack traces, noms de fichiers, requetés SQL). Journaliser les erreurs en détail cote serveur. Afficher des messages d'erreur generiques cote client.
Authentification et sessions	Utiliser des bibliothèques d'authentification éprouvées (ne pas réinventer la roue). Générer des tokens de session aléatoires et suffisamment longs. Invalider les sessions a la déconnexion et apres expiration. Régénérer le token de session apres connexion.
Gestion des dependances	Maintenir un fichier de dépendances a jour (package.json, requirements.txt, pom.xml). Spécifier les versions exactes des dépendances. Scanner regulierement les vulnérabilités avec npm audit, pip-audit ou mvn dependency-check.
Revue de code	Intégrer les revues de sécurité dans le processus de pull request. Former les reviewers aux patterns de vulnérabilités courants. Utiliser des checklists de sécurité lors des revues. Documenter les décisions de sécurité.

Travaux pratiques :

Installation de l'environnement

Objectif :

Installer et configurer les outils indispensables pour réaliser tous les TP du module Sécurité des Applications :

- VM Kali Linux
- VM Ubuntu CLI Linux
- VM Windows 10/11
- Docker pour lancer les environnements de test vulnérables
- OWASP Juice Shop l'application cible de vos exercices pratiques
- Burp Suite Community pour intercepter et analyser les requêtes HTTP

Installation de l'environnement des TP

Kali Linux (attaquant)	Ubuntu + Docker (serveur vulnérable)
▪ Nmap ▪ Burp Suite ▪ OWASP ZAP ▪ Wireshark	▪ DVWA :8081 ▪ WebGoat :8080 ▪ WebWolf :9090 ▪ Juice Shop :3000



TP: Audit de sécurité informatique

Objectif :

Analyse des vulnérabilités des réseaux, des systèmes et des applications web

- Nmap pour scanner les ports et services
- Nikto pour identifier les vulnérabilités web,
- Wireshark pour capturer et analyser le trafic réseau
- Burp Suite aide à l'interception et analyser les paramètres envoyés par une application web